AICORE AGENCY

Documento estratégico y técnico · Confidencial

AicoreLink

Plataforma Agéntica Multicanal · WhatsApp + Voz + Email + RRSS

Plan integral de negocio y arquitectura técnica

Multi-tenant · Feature flags por cliente · Dashboard con métricas

Construido sobre el stack de AiCoreMed (Next.js, PostgreSQL, n8n, Docker Swarm)

Preparado para: Leo · Aicore Agency · Santiago, Chile

Julio 2026

aicorebots.com

1. Resumen ejecutivo

Este documento define el plan completo -de negocio y tecnico- para construir **AicoreLink**, una plataforma agentica multicanal (WhatsApp texto y voz, correo electronico y redes sociales) que se instala en tu VPS actual y se activa por modulos segun lo que cada clinica contrate. Se apoya 100% en el stack que ya tienes en produccion: Next.js, PostgreSQL multi-tenant, n8n, Docker Swarm + Dokploy, Traefik, Redis, Cloudflare, y tu experiencia ya construida con Hermes Agent, Evolution API y OpenRouter.

El documento cubre: (1) el modelo de negocio y de precios por modulos activados, (2) una comparativa actualizada de proveedores de WhatsApp -incluyendo por que Evolution API no debe ser tu canal de produccion para clinicas-, (3) la arquitectura tecnica completa, (4) el diseno detallado de multi-tenancy sobre tu Postgres actual, (5) el plan de desarrollo por fases con entregables tecnicos concretos, y (6) un prompt de construccion detallado para cada fase, listo para usar en OpenCode con GLM 5.2.

Decisiones clave recomendadas en este documento:

- **Nombre:** AicoreLink (ver seccion 2 para el razonamiento y alternativas).
- **Modelo de cobro:** hibrido - fee de implementacion unico + suscripcion mensual por modulo activo (no pago unico puro).
- **Multi-tenancy:** base de datos compartida con **Row Level Security (RLS)** de PostgreSQL + tabla de *feature flags* por tenant, en vez de una base de datos por cliente.
- **WhatsApp de produccion:** Meta Cloud API directa (o via un BSP liviano como 360dialog), reservando Evolution API unicamente para staging/pruebas internas - nunca conectado a un numero real de un cliente pagante.
- **Orquestacion IA:** Ollama/Gemma3 local para clasificacion y tareas baratas, GLM 5.2 / DeepSeek via OpenRouter para razonamiento complejo - mismo patron que ya usas.

2. Nombre de la aplicacion

El producto se llama **AicoreLink**. Encaja bien con la linea de marca ya establecida (Aicore Agency, AiCoreMed) y el nombre "Link" comunica con precision lo que hace el sistema: conectar/vincular los distintos canales de comunicacion de la clinica (WhatsApp, voz, email, RRSS) con un unico agente y un unico panel de control.

- **Aicore** mantiene la continuidad de marca con AiCoreMed y con Aicore Agency como desarrollador.
- **Link** es directo y facil de explicar a una clinica no tecnica: "conecta todos tus canales de atencion en un solo lugar".
- El dominio **aicorebots.com** que ya posees permite alojar el dashboard naturalmente como *link.aicorebots.com*, sin comprar dominio nuevo.
- Se recomienda usar el prefijo AICORELINK_ para todas las variables de entorno propias del proyecto, y el prefijo de servicio aicorelink-* para los contenedores Docker, de forma que convivan sin choques con los servicios ya desplegados de AiCoreMed en el mismo VPS.

En el resto del documento se usa **AicoreLink** como nombre del producto.

3. Proveedores de WhatsApp: comparativa y recomendacion

Ya tienes Evolution API funcionando en tu VPS, y mencionas ManyChat/Trello como referencia de "herramientas estables y no muy costosas". Vale aclarar antes de comparar: **ManyChat** es una plataforma no-code de marketing/broadcast (pensada para creadores y marketing, no para correr tu propio agente IA con tu base de datos), y **Trello** es un gestor de tareas Kanban - ninguno de los dos es un reemplazo de infraestructura de mensajeria como Evolution API. La comparacion real es entre **APIs no oficiales** (Evolution API, Baileys, WAHA, WPPConnect) y **la API oficial de Meta**, accedida directo o via un BSP (Business Solution Provider).

3.1 Por que Evolution API es riesgoso para produccion comercial

Evolution API (y las librerias en las que se apoya, como Baileys) funcionan emulando el protocolo de WhatsApp Web: no son un canal oficial de Meta. Esto tiene tres consecuencias directas para un negocio que va a cobrar a clinicas por el servicio:

- **Riesgo de baneo real y medido:** reportes de 2026 hablan de ventanas de baneo de 2 a 8 semanas para herramientas que reconstruyen el protocolo (Baileys, WAHA, Evolution API, whatsmeow), especialmente si el numero envia volumen alto o mensajes a contactos que no iniciaron la conversacion.
- **Fragilidad ante cambios de Meta:** cuando Meta actualiza el protocolo de WhatsApp Web (ocurre varias veces al ano), el servicio se cae hasta que la comunidad libera un parche. Eso es un SLA que no controlas - inaceptable si le vendes disponibilidad a una clinica.
- **Carga operativa oculta:** sesiones de Baileys que se caen, reconexion manual, y que el numero de WhatsApp del cliente dependa de la salud de tu VPS (justo el tipo de incidente que ya viste con el error de Ollama/OOM) - ahora multiplicado por cada clinica conectada.

Ademas, para AiCoreMed el dato en juego es informacion clinica de pacientes: usar un canal no oficial para eso agrega un riesgo reputacional y de cumplimiento que va mas alla de la mensajeria.

3.2 Que cambio en el pricing de Meta que te conviene saber

- Meta paso de cobrar por *ventana de conversacion* de 24 horas a cobrar **por mensaje de plantilla enviado** (categorias Marketing, Utility, Authentication) desde mediados de 2025. Las conversaciones de **Service** (cuando el paciente escribe primero y tu respondes dentro de esa ventana) siguen siendo gratis.
- Esto favorece exactamente el patron de uso de una clinica: la mayoria de interacciones (agendar hora, confirmar, responder dudas) las inicia el paciente -> son gratis para Meta. Solo pagas por recordatorios/campanas que inicias tu (Utility/Marketing).
- Desde junio de 2026 Meta agrego **CLP (peso chileno)** como moneda de facturacion nativa, junto con ARS, COP, PEN, MXN ya disponibles - facturacion mas simple para tus clientes en Chile, Argentina y Mexico.
- El acceso a la Cloud API en si es gratuito; lo que se paga es el uso (por mensaje/categoria/pais) mas, si usas un BSP, su fee de plataforma.

3.3 Comparativa de opciones

Opcion	Tipo	Costo aproximado	Estabilidad / riesgo	Uso recomendado
Meta Cloud API directa	Oficial	Software: gratis. Se paga por mensaje segun categoria/pais a Meta	Maxima. Cero riesgo de baneo por protocolo	Produccion real, mejor costo-control a mediano plazo dado que ya tienes el stack tecnico
360dialog	BSP oficial (capa fina sobre Meta)	Fee de plataforma bajo + tarifa Meta pass-through	Alta	Si quieres production-ready mas rapido, sin construir tu propia capa de verificacion/plantillas
Twilio	BSP oficial	Fee + markup por mensaje encima de Meta (mas caro)	Alta	Solo si ya usas Twilio para otra cosa (voz/SMS) y prefieres un solo proveedor
Wati / Interakt / AiSensy / GreenTick	BSP oficial + CRM no-code	Planes desde ~USD 19-49/mes + tarifa Meta	Alta	No aplica a tu caso: ya tienes tu propio dashboard, pagarias por funciones que no usarias
Chatwoot (self-hosted) + Cloud API/360dialog	Bandeja open-source + canal oficial	Software gratis, solo hosting	Alta	Utilidad si en el futuro quieres bandeja compartida IA + agentes humanos
Evolution API (tu VPS actual)	No oficial (Baileys)	Gratis, solo costo de VPS	Riesgo de baneo (2-8 semanas), fragil ante updates de Meta	Solo staging/demos internas. Nunca conectado a un numero de un cliente pagante
WAHA / WPPConnect / Baileys puro	No oficial	Gratis - bajo	Mismo riesgo que Evolution API	Alternativas tecnicas equivalentes, mismo veredicto
WasenderAPI / Whapi.Cloud / UltraMsg	No oficial, cloud-hosted (sin VPS propio)	Desde ~USD 6-15/mes por sesion	Riesgo de baneo persiste, pero sin mantener servidor	Aceptable solo para MVP interno de bajisimo riesgo, nunca para datos de salud

Recomendacion final: migra el canal de produccion a Meta Cloud API directa (ya hiciste la evaluacion tecnica de esto). Mantén Evolution API corriendo en un stack de staging separado -util para probar flujos de n8n rapido sin gastar en plantillas- pero nunca lo conectes al numero real de una clinica que paga. Si en algun momento quieres delegar la parte de verificacion de Meta Business y plantillas para ir mas rapido con nuevos clientes, 360dialog es la opcion mas barata dentro de los BSP oficiales.

4. Arquitectura tecnica general

AicoreLink se construye como una extension de tu stack actual, no como un sistema paralelo. La idea central: un **nucleo unico** (base de datos, dashboard, motor de agente) sirve a todos los tenants (clinicas), y cada **canal** (WhatsApp texto, WhatsApp voz, Email, RRSS) es un worker independiente que se activa o desactiva por tenant sin afectar a los demas.

4.1 Componentes principales

Componente	Funcion	Tecnologia
Dashboard (app web)	Panel cliente + panel admin/dev, login, toggles de features, metricas	Next.js (App Router) + Tailwind, mismo patron que AiCoreMed
API Core	Autenticacion, tenants, entitlements, facturacion, metering de uso	Next.js API routes / route handlers + Prisma o Drizzle ORM
Base de datos	Tenants, usuarios, canales, credenciales, conversaciones, metricas	PostgreSQL (misma instancia que AiCoreMed, esquema separado)
Orquestador de flujos	Logica de cada canal: recibir, clasificar, responder, escalar	n8n (workflows genericos parametrizados por tenant_id)
Motor de IA	Clasificacion barata + respuesta conversacional + voz	Ollama/Gemma3 (local, barato) + GLM 5.2 / DeepSeek via OpenRouter (razonamiento)
Voz (STT/TTS)	Transcribir audio entrante, generar audio de respuesta	Whisper (local via Ollama-compatible runner) + ElevenLabs (ya lo usas) o Coqui/local TTS para bajar costo
Canal WhatsApp	Envio/recepcion de mensajes y audio	Meta Cloud API (produccion), Evolution API (solo staging)
Canal Email	Lectura/clasificacion/respuesta de correo	Gmail API / IMAP generico + n8n
Canal RRSS	Bandeja unificada de mensajes directos	Meta Graph API (Instagram + Messenger)
Cache / colas	Sesiones, rate limiting, colas de trabajo entre canal y agente	Redis (ya lo tienes)
Reverse proxy / TLS	Enrutamiento y certificados	Traefik + Cloudflare (ya lo tienes)
Orquestacion de contenedores	Despliegue y escalado de cada worker de canal	Docker Swarm + Dokploy (ya lo tienes)

4.2 Flujo de un mensaje (ejemplo WhatsApp texto)

- 1. El paciente escribe al numero de WhatsApp de la clinica -> llega al webhook unico de Meta Cloud API.
- 2. El webhook identifica el phone_number_id del mensaje y lo resuelve a un tenant_id en Postgres (tabla channel_credentials).
- 3. n8n consulta tenant_channels: ¿este tenant tiene **whatsapp_text** habilitado? Si no, responde con mensaje generico o corta el flujo.
- 4. Si esta habilitado, se carga el contexto del tenant (tono, horarios, base de conocimiento propia de la clinica) y se envia al motor de IA (Gemma3 para triage rapido; GLM 5.2 via OpenRouter si la consulta es compleja).
- 5. La respuesta se envia de vuelta por Meta Cloud API, y se registra en tenant_usage (tokens, costo estimado, canal) para metricas y facturacion.
- 6. Si la conversacion requiere un humano (por ejemplo, agendamiento complejo o queja), se marca escalado=true y aparece en el dashboard de la clinica.

El mismo patron (webhook unico -> resolver tenant -> chequear feature flag -> ejecutar -> registrar uso) se repite para email y RRSS, cambiando solo el paso 1 (como se identifica el tenant: por cuenta de correo, por page_id de Instagram, etc.) y el canal de salida.

---

5. Multi-tenancy: diseno detallado sobre tu stack actual

Esta es la decision arquitectonica mas importante del proyecto porque de ella depende que puedas escalar a decenas de clinicas sin multiplicar tu carga operativa. Hay tres modelos posibles; se explican los tres y se justifica la recomendacion.

5.1 Opciones de aislamiento de datos

Modelo	Como funciona	Ventaja	Desventaja para tu caso
Base de datos por tenant	Cada clinica tiene su propia base Postgres	Aislamiento total, facil de vender como "tus datos en tu propia base"	Migraciones x N bases, backups x N, no escala bien operado por una persona en un solo VPS
Schema por tenant	Una base Postgres, un schema SQL distinto por clinica	Aislamiento medio, mejor que N bases	A partir de ~20-30 tenants la gestion de migraciones y conexiones se vuelve compleja de mantener solo
Esquema compartido + tenant_id + RLS	Una base, una sola copia de tablas, cada fila marcada con tenant_id, PostgreSQL filtra automaticamente por tenant	Una sola migracion para todos, operacion simple, escala a cientos de tenants sin cambiar arquitectura	Requiere disciplina: toda query nueva debe respetar el contexto de tenant (mitigado por RLS a nivel de DB, no solo en la app)

Recomendacion: esquema compartido + tenant_id + Row Level Security (RLS). Es el modelo que usan la mayoria de SaaS B2B de este tamano (Notion, Linear, etc. en sus inicios) y es el que mejor encaja con que administras la infraestructura tu mismo desde un solo VPS. Ademas es coherente con lo que ya describe tu arquitectura actual de AiCoreMed ("PostgreSQL con arquitectura multi-tenant"): aqui se endurece ese diseno agregando RLS real a nivel de base de datos, en vez de confiar solo en que el codigo de la aplicacion filtre correctamente por tenant_id.

5.2 Por que RLS y no solo filtrar en el codigo

Si solo filtras por WHERE tenant_id = ? en cada query de la aplicacion, un solo bug o un endpoint nuevo que olvide ese filtro puede filtrar datos de una clinica a otra -inaceptable con datos clinicos. RLS mueve esa regla a la base de datos: aunque la aplicacion tenga un bug, Postgres no devuelve filas de otro tenant porque la politica de seguridad esta a nivel de motor, no de aplicacion.

5.3 Esquema de base de datos propuesto

```
-- Tenants (clinicas)
CREATE TABLE tenants (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          TEXT NOT NULL,
  slug          TEXT UNIQUE NOT NULL,       -- usado en subdominio o login
  plan          TEXT NOT NULL DEFAULT 'basico',
  status        TEXT NOT NULL DEFAULT 'activo',  -- activo | suspendido | trial
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- Modulos/canales activados por tenant (el corazon del "feature flag")
CREATE TABLE tenant_channels (
```

```
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  channel       TEXT NOT NULL CHECK (channel IN
                  ('whatsapp_text','whatsapp_voice','email','social')),
  enabled       BOOLEAN NOT NULL DEFAULT false,
  config        JSONB DEFAULT '{}',         -- tono, horarios, reglas de escalado, etc.
  activated_at  TIMESTAMPTZ,
  UNIQUE(tenant_id, channel)
);

-- Credenciales por canal y tenant (encriptadas a nivel de aplicacion antes de guardar)
CREATE TABLE channel_credentials (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),
  channel           TEXT NOT NULL,
  provider          TEXT NOT NULL,          -- meta_cloud_api | gmail | instagram | etc.
  external_id       TEXT,                   -- phone_number_id, page_id, email_account
  encrypted_secret  TEXT NOT NULL,
  created_at        TIMESTAMPTZ DEFAULT now()
);

-- Usuarios del dashboard (roles por tenant)
CREATE TABLE tenant_users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  user_id     UUID NOT NULL REFERENCES users(id),
  role        TEXT NOT NULL DEFAULT 'cliente'  -- cliente | admin_clinica | dev_aicore
);

-- Metrica y metering de uso (para dashboard y para facturar consumo real)
CREATE TABLE tenant_usage (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  channel       TEXT NOT NULL,
  period        DATE NOT NULL,              -- primer dia del mes
  messages_in   INT DEFAULT 0,
  messages_out  INT DEFAULT 0,
  tokens_used   INT DEFAULT 0,
  cost_estimate NUMERIC(10,2) DEFAULT 0,
  UNIQUE(tenant_id, channel, period)
);

-- Ejemplo de politica RLS sobre una tabla de datos operativos (conversaciones)
ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON conversations
  USING (tenant_id = current_setting('app.current_tenant')::UUID);

-- En cada conexion/request, la app fija el tenant activo antes de consultar:
-- SET app.current_tenant = '<uuid-del-tenant>';
```

5.4 Como se resuelve el tenant en cada canal

Canal	Como se identifica el tenant	Nota
Dashboard web	Sesion de login (JWT con tenant_id embebido)	Un solo dominio de dashboard (app.aicorebots.com); el tenant se resuelve por sesion, no por subdominio -> mas simple de operar
WhatsApp (Meta Cloud API)	phone_number_id del webhook -> tabla channel_credentials	Un solo webhook de Meta para todos los tenants; el numero de telefono de cada clinica se asocia a su tenant_id

Canal	Como se identifica el tenant	Nota
Email	Cuenta de correo que recibe el mensaje (IMAP/Gmail account)	Cada clinica conecta su propia cuenta o usa un alias dedicado
RRSS (Instagram/Messenger)	page_id / instagram_business_id del webhook de Meta Graph API	Mismo patron que WhatsApp: un webhook, resolucion por ID de pagina

5.5 n8n en modo multi-tenant (no un workflow por cliente)

El error mas comun al escalar con n8n es duplicar el workflow por cada cliente nuevo. Eso se vuelve inmantenible rapido. La alternativa correcta:

- **Un solo workflow por canal** (whatsapp, email, social), no uno por tenant.
- El primer nodo del workflow resuelve tenant_id a partir del payload entrante (phone_number_id, email, page_id).
- Un nodo Postgres/HTTP Request consulta tenant_channels y config antes de continuar - si el canal esta apagado para ese tenant, el flujo corta ahi.
- Toda configuracion especifica de la clinica (tono, horarios, reglas de escalado) vive en la columna config JSONB, no en nodos distintos de n8n - asi el workflow es identico para todos y solo cambia el dato.
- Al final del flujo, un nodo escribe en tenant_usage para métricas y billing, sin importar de que tenant vino el mensaje.

Esto significa que agregar un cliente nuevo es una operacion de **datos** (insertar filas en tenants, tenant_channels, channel_credentials), no una operacion de **codigo** (crear un workflow nuevo). Esa es la diferencia entre un negocio que escala y uno que no.

5.6 Enrutamiento con Traefik

Se recomienda **un solo dominio de dashboard** (por ejemplo *link.aicorebots.com*) con el tenant resuelto por login, en vez de un subdominio por clinica. Es el mismo modelo de Slack o Notion (workspace por login, no por subdominio) y evita la complejidad de emitir certificados wildcard y reglas de Traefik por cada cliente nuevo. Si en una fase futura quieres ofrecer *white-label* (subdominio propio por clinica), se puede agregar despues como una capa opcional sin rehacer el modelo de datos, ya que el tenant_id ya esta desacoplado del dominio.

---

6. Sistema de feature flags / entitlements

Este sistema es lo que permite activar/desactivar WhatsApp, voz, email o RRSS por cliente desde el panel, sin tocar codigo. Se construye sobre la tabla tenant_channels ya definida.

6.1 Middleware de verificacion

```
// lib/entitlements.ts
export async function hasFeature(tenantId: string, channel: Channel) {
  const row = await db.tenantChannels.findFirst({
    where: { tenantId, channel, enabled: true }
  });
  return !!row;
}

// Uso en un endpoint o antes de invocar un workflow de n8n
if (!(await hasFeature(tenantId, "whatsapp_voice"))) {
  return respondFeatureDisabled(); // mensaje generico o webhook 200 sin accion
}
```

6.2 Panel cliente vs. panel developer

Vista	Quien la ve	Que incluye
Panel cliente	Duenos/administrativos de la clinica	Toggles de SUS modulos contratados, metricas (mensajes procesados, tiempo de respuesta, % resuelto por IA vs escalado), configuracion de tono, horarios de atencion, plantillas de respuesta
Panel developer/admin (tu)	Tu equipo (Aicore Agency)	Estado de salud de cada worker (WhatsApp/Email/RRSS) por tenant, consumo de tokens y costo real por tenant, logs de n8n, activar/desactivar modulos sin depender del cliente, override de modelo de IA por tenant, alertas de infraestructura (ej. caida de Ollama, error de webhook)

Importante: el toggle que ve el cliente en su panel no debe escribir directamente enabled=true sin pasar por el flujo de facturacion - la activacion real de un modulo pagado debe quedar condicionada a que el pago/plan lo permita (verificado en el backend, no confiado al frontend).

---

7. Plan de desarrollo por fases (detallado)

Cada fase entrega algo vendible o operable por si solo -no se espera a tener todo el sistema listo para empezar a facturar.

Fase 0 - Fundacion (2-3 semanas)

Objetivo: Tener la base de datos, autenticacion y facturacion listas antes de construir cualquier canal.

Entregables tecnicos:

- Esquema de base de datos completo (tenants, tenant_channels, channel_credentials, tenant_users, tenant_usage) con RLS activado.
- Autenticacion multi-tenant (NextAuth o similar) con JWT que incluya tenant_id y rol.
- Dashboard shell (layout, navegacion, panel cliente vacio + panel admin vacio).
- Integracion de cobro recurrente (Stripe si facturas en USD, o Flow/Webpay/MercadoPago si cobras en CLP directo).

Entregable comercial: Ninguno aun vendible, pero es la base que evita retrabajo en las fases siguientes.

Fase 1 - WhatsApp texto (MVP vendible) (3-4 semanas)

Objetivo: Primer modulo facturable: agente de WhatsApp por texto funcionando en produccion con Meta Cloud API.

Entregables tecnicos:

- Migracion de Twilio sandbox / Evolution API a Meta Cloud API en produccion para el primer cliente piloto.
- Workflow n8n generico de WhatsApp texto (resolver tenant -> feature flag -> IA -> respuesta -> log de uso).
- Panel cliente v1: activar/desactivar WhatsApp, ver conversaciones recientes, metricas basicas.
- Base de conocimiento por clinica (horarios, servicios, preguntas frecuentes) editable desde el panel.

Entregable comercial: Plan Basico: WhatsApp con IA, listo para vender a las clinicas piloto que ya tienes en proceso de cierre.

Fase 2 - WhatsApp voz (3-4 semanas)

Objetivo: Agregar manejo de notas de voz entrantes y respuestas en audio.

Entregables tecnicos:

- Pipeline STT (Whisper local o API) para transcribir audio entrante desde WhatsApp.
- Pipeline TTS (ElevenLabs, que ya usas, o alternativa local para bajar costo variable) para generar audio de respuesta.
- Ajuste del workflow n8n para manejar tipo de mensaje (texto vs audio) y latencia aceptable (objetivo menor a 6-8 segundos).
- Feature flag independiente whatsapp_voice, facturable como upgrade sobre WhatsApp texto.

Entregable comercial: Plan Intermedio: WhatsApp texto + voz.

Fase 3 - Email con IA (2-3 semanas)

Objetivo: Triage y respuesta automatica/sugerida de correo entrante de la clinica.

Entregables tecnicos:

- Conexion via Gmail API o IMAP generico por tenant (credenciales encriptadas en channel_credentials).
- Workflow n8n de clasificacion (consulta, reclamo, agendamiento, spam) y respuesta sugerida o automatica segun configuracion del tenant.
- Reglas de escalado a un humano cuando la confianza del modelo es baja.
- Metricas de email en el panel: correos procesados, tasa de respuesta automatica, tiempo promedio de respuesta.

Entregable comercial: Modulo Email disponible como add-on independiente o combinado con WhatsApp.

Fase 4 - RRSS (Instagram / Messenger) (2 semanas)

Objetivo: Bandeja unificada de mensajes directos de redes sociales con el mismo agente.

Entregables tecnicos:

- Integracion con Meta Graph API para Instagram Direct y Messenger.
- Reuso del mismo motor de agente y las mismas tablas de configuracion (config JSONB) que WhatsApp/Email.
- Panel cliente: bandeja unificada de RRSS con metricas propias.

Entregable comercial: Modulo RRSS, ultimo componente para ofrecer el plan Completo (los 4 canales).

Fase 5 - Metricas avanzadas y utilidades developer (2 semanas)

Objetivo: Dar visibilidad operativa fina, tanto al cliente como a ti como operador de la infraestructura.

Entregables tecnicos:

- Dashboard de costo real por tenant (tokens consumidos x tarifa de OpenRouter/Meta) vs. lo facturado - margen por cliente visible.
- Alertas automaticas de infraestructura (caida de Ollama, error de webhook, contenedor reiniciado) enviadas a un canal interno (Telegram, ya usas ese patron con Hermes Agent).
- Panel admin con logs de n8n embebidos o enlazados por tenant para debugging rapido.

Entregable comercial: No es un modulo facturable directo, pero reduce tu carga operativa y protege el margen a medida que sumas clientes.

Fase 6 - Self-serve (opcional, mediano plazo) (3-4 semanas)

Objetivo: Que un cliente nuevo pueda contratar, activar y pagar modulos sin que intervengas manualmente.

Entregables tecnicos:

- Flujo de onboarding self-serve: registro, seleccion de plan, checkout, activacion automatica de flags tras pago confirmado (webhook de Stripe/Flow).
- Wizard de conexion de WhatsApp Business (verificacion Meta) y de cuenta de correo dentro del propio panel.
- Documentacion y ayuda contextual en el dashboard para reducir soporte manual.

Entregable comercial: Habilita crecimiento sin que cada cliente nuevo consuma tu tiempo de implementacion.

Tiempo total estimado (fases 0 a 5, sin self-serve): aproximadamente 14-18 semanas trabajando de forma consistente con apoyo de IA para codificacion (OpenCode + GLM 5.2). Las fases 1 y 2 son las que definen si el negocio es viable, por lo que conviene validarlas con clientes piloto reales antes de construir email y RRSS.

---

8. Infraestructura (Docker Swarm sobre tu VPS actual)

AicoreLink se despliega como servicios adicionales dentro del mismo stack de Dokploy/Traefik que ya administras, no como infraestructura aparte.

```
# docker-compose.aicorelink.yml (esqueleto, se despliega via Dokploy)
version: "3.8"
services:

  aicorelink-dashboard:
    image: aicorelink/dashboard:latest
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - NEXTAUTH_SECRET=${NEXTAUTH_SECRET}
    deploy:
      replicas: 2
    networks: [traefik-public, internal]
    labels:
      - traefik.http.routers.aicorelink.rule=Host(`link.aicorebots.com`)

  aicorelink-whatsapp-worker:
    image: aicorelink/whatsapp-worker:latest
    environment:
      - META_CLOUD_API_TOKEN=${META_CLOUD_API_TOKEN}
      - DATABASE_URL=${DATABASE_URL}
      - OPENROUTER_API_KEY=${OPENROUTER_API_KEY}
    deploy:
      replicas: 1
    networks: [internal]

  aicorelink-email-worker:
    image: aicorelink/email-worker:latest
    environment:
      - DATABASE_URL=${DATABASE_URL}
    networks: [internal]

  aicorelink-social-worker:
    image: aicorelink/social-worker:latest
    environment:
```

```
      - DATABASE_URL=${DATABASE_URL}
    networks: [internal]

  # n8n, postgres, redis y ollama: reutilizas los servicios ya desplegados

networks:
  traefik-public:
    external: true
  internal:
    external: true
```

Cada worker de canal escala y se reinicia de forma independiente. Si el worker de email falla, WhatsApp de todos los tenants sigue funcionando -aislamiento a nivel de infraestructura, ademas del aislamiento a nivel de datos que da RLS.

---

9. Seguridad y cumplimiento

- **Datos de salud:** AiCoreMed maneja informacion clinica; toda credencial (tokens de Meta, contrasenas de correo) debe guardarse encriptada en channel_credentials, nunca en texto plano ni en variables de entorno compartidas entre tenants.
- **RLS como defensa en profundidad:** incluso si un endpoint de la API tiene un bug de autorizacion, la politica de RLS en Postgres impide que devuelva filas de otro tenant.
- **Principio de menor privilegio en n8n:** las credenciales de canal se inyectan por tenant en tiempo de ejecucion (via nodo que lee de channel_credentials), no se guardan credenciales fijas por workflow.
- **Auditoria:** registrar quien activo/desactivo un modulo y cuando (tabla de auditoria simple), util tanto para soporte como para disputas de facturacion.
- **Verificacion de Meta Business:** el proceso de verificacion de negocio y numero de WhatsApp debe hacerse a nombre de cada clinica o de Aicore Agency como partner, segun el modelo comercial que definas con tus clientes -conviene decidirlo antes de escalar a mas de 2-3 clientes.

---

10. Modelo de precios: plan mensual vs. pago unico

Recomendacion: modelo hibrido - fee de implementacion unico (cobra tu tiempo de setup) + suscripcion mensual por modulo activo (cobra el servicio continuo). Un pago unico puro no cubre tus costos variables recurrentes (tokens de OpenRouter, mensajeria de Meta, hosting) y no refleja que el valor real para la clinica es que el agente siga funcionando y mejorando mes a mes.

10.1 Por que no pago unico puro

- Tus costos variables (OpenRouter/GLM 5.2, tarifas de Meta por mensaje de plantilla, ElevenLabs para voz) crecen con el uso de cada clinica - un pago unico no los cubre pasado cierto volumen.
- El valor que compra la clinica no es solo software: es un agente que sigue funcionando, se actualiza y tiene soporte - eso es naturaleza de servicio recurrente, no de producto que se entrega una vez.
- Un pago unico incentiva mal: te conviene mas tener clientes contentos y activos mes a mes que cobrar una vez y desentenderte del soporte continuo.

10.2 Estructura de planes por modulos activos

Plan	Modulos incluidos	Logica de precio
Basico	WhatsApp texto	Precio base del modulo mas simple
Intermedio	WhatsApp texto + voz, o WhatsApp + Email (a eleccion del cliente)	Base + ~60-80% del valor de un modulo adicional (no se cobra el segundo modulo al 100% para incentivar el upgrade)
Completo	WhatsApp texto + voz + Email + RRSS	Base + modulos restantes con descuento de bundle (10-15% respecto de contratarlos por separado)

No se incluyen cifras en pesos chilenos en este documento porque fijar el numero correcto depende de validar percepcion de valor con tus clinicas piloto -tienes una ventaja aqui: ya construiste el paquete "Aicore Captacion 360", que te da un ancla de precio percibido en el mismo segmento de clientes. Se recomienda correr 2-3 conversaciones de pricing con clinicas reales antes de fijar el numero final, y usar como referencia el costo variable real medido en la Fase 1 (tokens + mensajes de Meta por clinica piloto) para poner un piso que garantice margen.

10.3 Fee de implementacion (cobro separado, no diluido en el mensual)

Cobra el fee de setup aparte del plan mensual. Cubre trabajo puntual que no se repite: verificacion de Meta Business, conexion de cuentas de correo, configuracion inicial de tono y base de conocimiento de la clinica. Esto protege tu margen si un cliente cancela en el mes 2 -ya cobraste el trabajo de implementacion independientemente de cuanto tiempo se quede suscrito.

10.4 Metering de uso como salvaguarda

Ademas del plan fijo, la tabla tenant_usage te permite detectar clientes cuyo volumen real de mensajes/tokens excede lo previsto para su plan (por ejemplo, una clinica grande usando el plan Basico con volumen de plan Completo). Esto te da datos objetivos para conversar un upgrade de plan en vez de subsidiar silenciosamente a los clientes de mayor uso con los ingresos de los demas.

---

11. Configuracion recomendada de OpenCode: MCP y skills

Antes del prompt de construccion (seccion 12), esto es lo que conviene tener configurado en OpenCode para que GLM 5.2 trabaje con contexto real de tu infraestructura en vez de solo generar codigo aislado. Verifica disponibilidad exacta de cada servidor MCP en tu version de OpenCode antes de asumir que esta activo.

MCP / herramienta	Para que sirve en este proyecto
MCP de PostgreSQL	Que el modelo pueda inspeccionar el esquema real de tu base (tablas de AiCoreMed) antes de proponer migraciones, evitando choques de nombres o tipos
MCP de filesystem / repo local	Navegar el repo consultorio-medico existente para reusar componentes de UI, patrones de autenticacion y convenciones ya establecidas
MCP de GitHub	Crear ramas, PRs y trackear el trabajo por fase (Fase 0, Fase 1, etc.) de forma ordenada
MCP de Docker / exec en el VPS	Validar que los nuevos servicios (workers de canal) levantan correctamente en Swarm antes de darlos por terminados

MCP / herramienta	Para que sirve en este proyecto
Skill/plantilla de n8n (si existe en tu OpenCode)	Generar los workflows JSON de n8n directamente importables, siguiendo el patron generico por canal descrito en la seccion 5.5

Si alguno de estos MCP no esta disponible en tu configuracion actual, el prompt de la seccion 12 esta escrito para funcionar igual de bien solo con acceso a filesystem y terminal (bash), que es el minimo que ya usas con OpenCode + GLM 5.2 en Swarm.

12. Prompts de construccion para OpenCode / GLM 5.2

En vez de un unico prompt generico, se entrega un archivo separado -**AicoreLink_Prompts_Por_Fase.md**- con **un prompt independiente y completo por cada fase** (Fase 0 a Fase 5), en linea con el plan de la seccion 7. Cada prompt de fase es autocontenido: incluye su propia estructura de carpetas, sus propios servicios de infraestructura, sus propias variables de entorno, el detalle de base de datos que corresponde a esa fase, y su criterio de aceptacion -no es un resumen, es la especificacion completa que el modelo necesita para ejecutar esa fase sin inventar supuestos.

Cada prompt de fase sigue esta misma estructura interna:

- Objetivo y alcance exacto de la fase (que si entra y que no entra).
- Estructura de carpetas y archivos nuevos que se crean en esa fase especifica.
- Infraestructura: servicios Docker nuevos o modificados, variables de entorno, redes, integracion con Traefik.
- Cambios de base de datos (tablas o columnas nuevas) con SQL/Prisma listo para usar.
- Especificacion funcional detallada (que hace cada endpoint, cada nodo de n8n, cada pantalla).
- Criterio de aceptacion verificable, para saber cuando la fase esta realmente terminada.

Se recomienda pegar **un prompt de fase a la vez** como contexto en OpenCode -no las seis fases juntas- para mantener el contexto acotado y poder revisar y aprobar cada entrega antes de pasar a la siguiente, que es tu flujo de trabajo habitual.

Anexo: glosario rapido

Termino	Significado en este documento
Tenant	Una clinica cliente de AicoreLink, con sus propios datos y modulos contratados
Feature flag / entitlement	Bandera en base de datos que indica si un modulo (canal) esta activo para un tenant
RLS (Row Level Security)	Mecanismo de PostgreSQL que filtra filas automaticamente segun el tenant activo en la sesion
BSP (Business Solution Provider)	Empresa autorizada por Meta para revender acceso a la API oficial de WhatsApp
Worker de canal	Servicio Docker independiente que procesa un canal especifico (WhatsApp, email, RRSS)
Metering	Medicion de uso real (mensajes, tokens) por tenant, usada para metricas y control de margen

Fin del documento. Los prompts de construccion, uno por cada fase, se entregan en el archivo adjunto *AicoreLink_Prompts_Por_Fase.md*.